

TaskFlow — Full-Stack Walkthrough

How the Next.js frontend links to the Spring API, &
how to **spot** Spring building blocks in the actual code

Written for someone who knows the words (DTO, repository, service...) but finds them
hard to point at in real code. We fix that with your project's own files.

Covers Phases 1–10 + the frontend · Generated 11 June 2026

Spring Boot API

Next.js + Tailwind

JWT

CORS

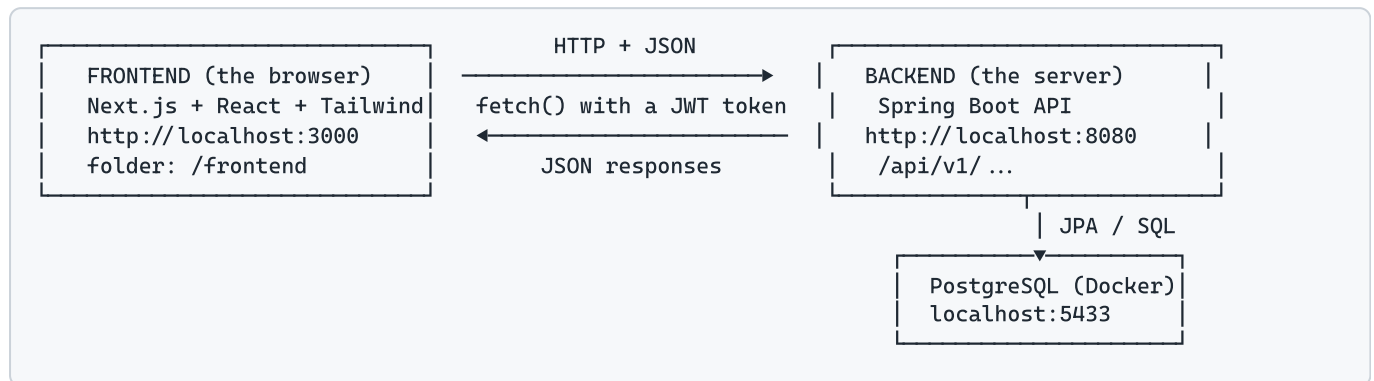
JPA

HOW TO GET THE MOST FROM THIS GUIDE

Part 4 is the one you asked for — a "spot-the-pattern" field guide: for each Spring concept it shows the *tell-tale sign* to look for, what it does, and **the exact file in your project** where it lives, with the real snippet. Parts 1–3 explain how the two apps (frontend + backend) talk to each other, traced through one real button click. Open the files side-by-side with this PDF as you read.

Part 1 — Two apps, one system

TaskFlow is really two programs that talk over HTTP.



- **Frontend** = what the user sees and clicks. It holds NO database access and NO secrets. It only knows how to call the API and render what comes back.
- **Backend** = the brain + the data. It enforces the rules (auth, ownership) and is the only thing that touches the database.

ANALOGY

The frontend is the **dining room** (menus, tables, what the customer touches). The backend is the **kitchen + storeroom** (the cooking rules and the ingredients). They communicate through a **serving hatch** — that hatch is HTTP, and the language passed through it is JSON.

Part 2 — The four things that make the link work

2.1 CORS — the browser's permission slip

Browsers block a page on one origin (`localhost:3000`) from calling a different origin (`localhost:8080`) unless the server explicitly allows it. That permission is **CORS**.

Where we allow it

`config/CorsConfig.java`

```
config.setAllowedOrigins(allowedOrigins);    // includes http://localhost:3000
config.setAllowedMethods(List.of("GET","POST","PUT","PATCH","DELETE","OPTIONS"));
config.setAllowedHeaders(List.of("*"));       // notably Authorization (our token)
```

Without this, every call from the frontend would fail in the browser with a CORS error — even though the API itself is fine. This is the #1 gotcha when a frontend "can't reach" a working backend.

2.2 JWT — the "I'm logged in" wristband

HTTP is stateless: the server forgets you between requests. So after login the server hands the browser a signed token (a **JWT**). The browser sends it back on every request to prove who it is.

1. User submits email+password → `POST /api/v1/auth/login`
2. Server verifies, returns a JWT ← `{ "token": "eyJ...", "user": { ... } }`
3. Frontend stores the token (`localStorage`)
4. Every later request carries it: `Authorization: Bearer eyJ...`
5. Server reads the token, knows the user, allows the request

2.3 The API client — the single bridge file

Instead of scattering `fetch()` calls everywhere, the frontend funnels them all through one helper. This is where the token gets attached and errors get unwrapped.

```
async function request<T>(path, options) {
  const headers = { ...options.headers };
  if (options.body) headers["Content-Type"] = "application/json";
  const token = getToken(); // from localStorage
  if (token) headers["Authorization"] = `Bearer ${token}`; // ← the link!
  const res = await fetch(`${BASE}${path}`, { ...options, headers });
  ...
}

export const apiToggleComplete = (id) =>
  request<Task>(`/tasks/${id}/complete`, { method: "PATCH" });
```

`BASE` = `http://localhost:8080/api/v1`. Every `apiXxx()` function maps 1:1 to a backend endpoint, so the frontend reads like a list of the API's capabilities.

2.4 The request/response cycle

A frontend action becomes an HTTP request, travels through the backend layers, and returns JSON the UI re-renders. Part 3 traces exactly that for one click.

Part 3 — One click, traced end-to-end

You click “**Done**” on a task. Here is every hop, with the real code at each.

#	Where (file)	What happens
1	<code>frontend/src/app/tasks/page.tsx</code>	The button's <code>onClick</code> calls <code>toggle(id)</code> → <code>await apiToggleComplete(id)</code> .
2	<code>frontend/src/lib/api.ts</code>	<code>request()</code> attaches <code>Authorization: Bearer <token></code> and does <code>fetch(PATCH /tasks/{id}/complete)</code> .
3	<code>config/CorsConfig.java</code>	Browser's CORS check passes (origin :3000 is allowed).
4	<code>auth/filter/JwtAuthFilter.java</code>	Reads the token, verifies its signature, extracts the user id, marks the request as that user.
5	<code>config/SecurityConfig.java</code>	Rule check: <code>/api/v1/**</code> needs auth → the request is authenticated → allowed.
6	<code>task/controller/TaskController.java</code>	<code>@PatchMapping("/{id}/complete")</code> runs; <code>@AuthenticationPrincipal</code> UUID <code>userId</code> supplies the caller; calls the service.
7	<code>task/service/TaskService.java</code>	<code>toggleComplete</code> : <code>getOwnedOrThrow</code> (404 if not yours), flips status, <code>applyStatus</code> updates <code>completedAt</code> , saves.
8	<code>task/repository/TaskRepository.java</code>	<code>save()</code> → Hibernate turns it into an SQL <code>UPDATE</code> .
9	PostgreSQL	The row is updated.
10	<code>task/mapper/TaskMapper.java</code>	Converts the saved entity → <code>TaskResponse</code> DTO (nesting the category).
11	back in <code>tasks/page.tsx</code>	The promise resolves; <code>load()</code> re-fetches; React re-renders the badge as DONE. No page reload.

THE TAKEAWAY

Frontend → **api.ts** → (CORS) → **filter** → **controller** → **service** → **repository** → DB, then a DTO comes back. Every feature is this same pipeline. Learn it once; recognise it everywhere.

Part 4 — ★ How to spot Spring building blocks in the code

For each concept: the **tell** (what to look for), what it does, and where it lives in YOUR project.

CONTROLLER

task/controller/TaskController.java

The tell: class annotated `@RestController` + `@RequestMapping("/api/v1/ ... ")`, with methods marked `@GetMapping` / `@PostMapping` / `@PutMapping` / `@PatchMapping` / `@DeleteMapping`.

Does: the front door for HTTP. Maps a URL + verb to a method, reads the request, returns the response. Contains *no* business rules — it delegates to a service.

```
@RestController
@RequestMapping("/api/v1/tasks")           // ← every method's URL starts here
public class TaskController {
    @PatchMapping("/{id}/complete")        // ← PATCH /api/v1/tasks/{id}/complete
    public TaskResponse complete(@AuthenticationPrincipal UUID userId,
                                @PathVariable UUID id) {
        return taskService.toggleComplete(userId, id); // ← hands off to the service
    }
}
```

Also notice `@AuthenticationPrincipal` (the logged-in user's id), `@PathVariable` (part of the URL), `@RequestBody` (the JSON body), `@RequestParam` (query params like `?status=`).

DTO (Data Transfer Object)

task/dto/TaskRequest.java, TaskResponse.java

The tell: a Java **record** living in a `dto/` package. **No** `@Entity`. Often has validation annotations (`@NotBlank`, `@Size`). It's the *shape of the JSON* in or out.

Does: defines exactly what the outside world sends/receives — deliberately separate from the database entity (so internal fields like a password hash can never leak).

```
public record TaskRequest(
    @NotBlank @Size(max = 255) String title, // ← validation rules
    String description,
    TaskStatus status, Priority priority,
    Instant dueDate, UUID categoryId
) {}
```

Rule of thumb: **Request** DTO = data coming IN; **Response** DTO = data going OUT. If it's a **record** and it's in `dto/`, it's a DTO.

ENTITY

task/entity/Task.java, user/entity/User.java

The tell: class annotated `@Entity` + `@Table(name="...")`, fields with `@Id`, `@Column`, `@ManyToOne`. Lives in an `entity/` package.

Does: one entity object = one row in a database table. This is the in-memory mirror of your data.

```
@Entity
@Table(name = "tasks")                // ← maps to the 'tasks' table
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    @Column(nullable = false) private String title;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "owner_id")
    private User owner;                // ← a relationship to another table
}
```

`@ManyToOne` = "many tasks belong to one user." That's a foreign key, expressed in Java.

REPOSITORY

task/repository/TaskRepository.java

The tell: an `interface` (not a class!) that `extends JpaRepository<Entity, IdType>`. It has method *signatures* but **no bodies** — you never write the implementation.

Does: all database reads/writes. Spring generates the actual SQL at startup from the method names.

```
public interface TaskRepository
    extends JpaRepository<Task, UUID> {                // ← gives save/findById/delete free
    Optional<Task> findByIdAndOwnerId(UUID id, UUID ownerId); // ← name → SQL WHERE
    long countByOwnerIdAndStatus(UUID ownerId, TaskStatus status);
}
```

"Derived query": Spring reads `findByIdAndOwnerId` and writes `WHERE id=? AND owner_id=?` for you. An interface extending `JpaRepository` is **always** a repository.

The tell: class annotated `@Service`, sitting in a `service/` package, often with `@Transactional` methods. It injects repositories/mappers in its constructor.

Does: the business rules — the "brain." Validates ownership, applies defaults, coordinates repository calls. The controller calls it; it calls the repository.

```
@Service
public class TaskService {
    private final TaskRepository taskRepository;    // ← injected dependency
    public TaskService(TaskRepository taskRepository, ...) { ... } // ← constructor injection

    @Transactional                                // ← all-or-nothing DB unit
    public TaskResponse toggleComplete(UUID ownerId, UUID id) {
        Task task = getOwnedOrThrow(ownerId, id); // ← the ownership RULE lives here
        ...
    }
}
```

If you're looking for "where does the actual logic happen?" — it's the `@Service`. Controllers are thin; services are where decisions are made.

The tell: a `@Component` in a `mapper/` package with methods like `toResponse(entity)` that build a DTO from an entity.

Does: translates between Entity (DB shape) and DTO (wire shape) in one reusable place.

```
@Component
public class TaskMapper {
    public TaskResponse toResponse(Task task) { // ← Entity in, DTO out
        return new TaskResponse(task.getId(), task.getTitle(), ...);
    }
}
```


CONFIG & @Bean

config/PasswordEncoderConfig.java

The tell: class annotated `@Configuration` , with methods annotated `@Bean` .

Does: hands Spring a ready-made shared object (a "bean") to inject elsewhere. Used for things you don't write as `@Service` / `@Component` (often library types).

```
@Configuration
public class PasswordEncoderConfig {
    @Bean                                // ← "store this object in the container"
    public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

GLOBAL ERROR HANDLER

common/exception/GlobalExceptionHandler.java

The tell: class annotated `@RestControllerAdvice` with methods annotated `@ExceptionHandler(SomeException.class)` .

Does: one global "catch block" for the whole API. Any exception a controller throws lands here and becomes a tidy JSON error.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ApiException.class)    // ← catches our NotFound/Conflict/etc.
    public ResponseEntity<ErrorResponse> handleApi(ApiException ex, HttpServletRequest req) {
    }
}
```

SPECIFICATIONS (dynamic filters)

task/spec/TaskSpecifications.java

The tell: static methods returning `Specification<Entity>` , combined with `Specification.allOf( ... )` in the service.

Does: builds `WHERE` -clause pieces at runtime so one endpoint supports many filter combinations (Phase 8).

```
public static Specification<Task> statusEquals(TaskStatus status) {
    if (status == null) return null;           // ← absent filter = skipped
    return (root, query, cb) → cb.equal(root.get("status"), status);
}
```

DEPENDENCY INJECTION — HOW TO SPOT IT EVERYWHERE

You'll see this pattern in every service/controller: a `private final` field + a constructor that receives it. You **never** see `new TaskService( ... )`. That's **dependency injection**: Spring creates one of each `@Service` / `@Component` / `@Bean` and passes them in automatically. When you see `private final X x;` set only in the constructor — that's an injected dependency.

Part 5 — How to read ANY feature (the recipe)

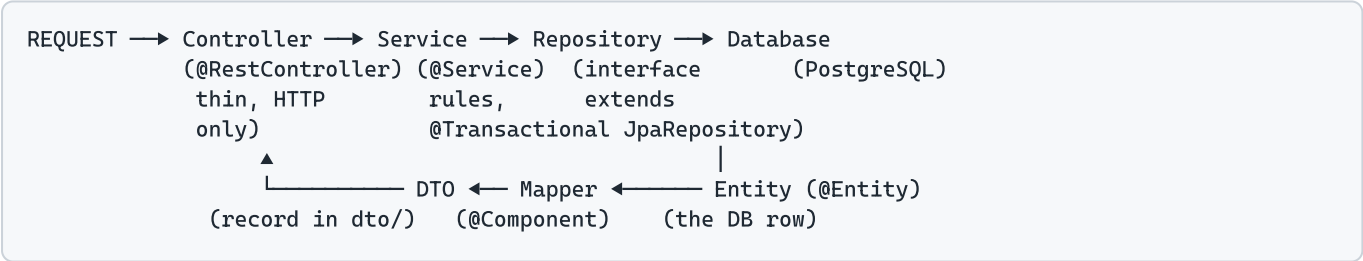
When you open a feature and feel lost, read its files in this order. Using **tasks** as the example:

Step	Layer	File	Question it answers
1	Controller	task/controller/TaskController.java	What URLs exist? What goes in/out?
2	DTOs	task/dto/TaskRequest, TaskResponse	What's the exact JSON shape?
3	Service	task/service/TaskService.java	What are the rules? (ownership, defaults, the toggle)
4	Repository	task/repository/TaskRepository.java	How is the DB queried?
5	Entity	task/entity/Task.java	What does one row look like + its relationships?
6	Mapper	task/mapper/TaskMapper.java	How does an entity become the response DTO?

WHY THIS ORDER

You follow the same path a request takes. By the time you reach the entity, you already know *why* each field matters because you saw it used in the controller and service. Top-down beats reading files alphabetically.

The whole backend, one screen



Part 6 — Annotation cheat-sheet

See one of these on a class/method/field? Here's what it means and which layer it signals.

Annotation	Layer / meaning
<code>@RestController</code>	Controller — handles HTTP, returns JSON.
<code>@RequestMapping</code> / <code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PutMapping</code> / <code>@PatchMapping</code> / <code>@DeleteMapping</code>	Controller — binds a URL + HTTP verb to a method.
<code>@PathVariable</code> / <code>@RequestParam</code> / <code>@RequestBody</code>	Controller — pull data from the URL path / query string / JSON body.
<code>@Valid</code>	Controller — run the DTO's validation rules before the method.
<code>@AuthenticationPrincipal</code>	Security — inject the logged-in user's id (from the JWT).
<code>@Service</code>	Service — business logic bean.
<code>@Transactional</code>	Service — wrap DB work in one all-or-nothing transaction.
<code>@Component</code>	Generic Spring-managed bean (e.g. mappers, the JWT filter).
<code>@Repository</code> / <code>extends JpaRepository</code>	Repository — database access.
<code>@Entity</code> / <code>@Table</code> / <code>@Id</code> / <code>@Column</code> / <code>@ManyToOne</code>	Entity — maps a class/field to a table/column/relationship.
<code>@Configuration</code> / <code>@Bean</code>	Config — defines beans (often library objects).
<code>@RestControllerAdvice</code> / <code>@ExceptionHandler</code>	Error handling — global catch-and-format.
<code>record ... { }</code> in <code>dto/</code>	DTO — the JSON shape (no annotation needed to be a DTO).

Part 7 — What we've built so far

Phase	What it added	Key files to recognise it
1–2	Project skeleton + DB schema (Flyway)	pom.xml, application*.yaml, db/migration/V1__init.sql
3	Entities + repositories + auditing	*/entity/*, */repository/*, common/audit
4	Security: BCrypt + JWT	config/SecurityConfig, auth/service/JwtService, auth/filter/JwtAuthFilter
5	Register/login + /users/me	auth/controller, auth/service/AuthService, user/*
6	Categories CRUD	category/* (full slice)
7	Tasks CRUD + complete-toggle	task/* (full slice)
8	Task filtering + pagination	task/spec/TaskSpecifications, common/dto/PageResponse
9	Consistent error handling	common/exception/*, common/dto/ErrorResponse
10	Analytics summary	analytics/*, aggregation queries in TaskRepository
—	Next.js + Tailwind frontend	frontend/src/app/*, frontend/src/lib/(api, auth, types)

Still to come: Phase 12 (deployment), Phase 11 (tests), Phase 13 (wrap-up).

Glossary

Term	Plain meaning
Origin	scheme + host + port, e.g. <code>http://localhost:3000</code> . CORS is about cross-origin calls.
CORS	Browser rule; the server must allow another origin to call it.
JWT	Signed token proving who you are; sent as <code>Authorization: Bearer ...</code> .
localStorage	Browser key-value store; we keep the token here between page loads.
fetch	The browser's function to make HTTP requests (used in <code>api.ts</code>).
Controller	Backend layer that handles HTTP and returns JSON.
Service	Backend layer holding the business rules.
Repository	Backend interface that reads/writes the database.
Entity	Java class mapped to a DB table (one object = one row).
DTO	The shape of JSON in/out; kept separate from the entity.
Mapper	Converts entity ↔ DTO.
Bean / DI	An object Spring creates and injects for you (you never <code>new</code> it).
Specification	A composable WHERE-clause piece for dynamic filtering.
Client / Server component (Next.js)	Client = runs in browser (interactive); Server = runs on server. Our app is client-rendered.
Context (React)	Shared state across components (we use it for the logged-in user — <code>auth.tsx</code>).

TaskFlow full-stack guide · backend at `localhost:8080/api/v1` · frontend at `localhost:3000` · Pair this with the per-file walkthrough (Phases 1–7) and the testing guide in [docs/](#) .